# MIE443H1S: Contest 2
# I Can Help You Clean: Pick and Place of Household Objects

**1.1 Objective:**
The goal of this contest is for the TurtleBot 4 to detect and pick up a manipulable object placed on top of the robot, navigate an environment to find and identify five scene objects placed at different locations in the environment, compare the manipulable object with the scene objects in order to place it in the correct garbage bin in front of the corresponding scene object, and return to its starting position when the robot is finished. The coordinates of the manipulable object, each scene object and the map of the environment will be provided to your team in advance. Your team will be in charge of programming the TurtleBot 4 to navigate itself to locations in the environment where it can view and identify each scene object, and then return to its starting location, all within a specified time limit. Team scores will be determined based on the ability to pick and place the manipulable object, the identification of the scene objects, and the ability of the robot to return to the start position.

**1.2 Requirements:**
The contest requirements are:

- The contest environment will be $4.87 \times 4.87$ m$^2$, Figure 1. The environment will consist of 5 scene objects, obstacles and 5 garbage bins. The 5 scene objects are common household objects: 1) a plastic cup, 2) a water bottle, 3) a clock, 4) a potted plant, and 5) a toy motorcycle. Each scene object has a garbage bin directly in front of it with an AprilTag attached to the bin. One manipulable object (i.e., a cup) will be placed on top of the Turtlebot 4 on its top plate by the Instructor/TAs on contest day. The coordinates of the manipulable object in the arm base frame, and the coordinates of the scene objects in the world frame, will be provided by the Instructor/TAs on contest day. A test set of object coordinates will be provided for development purposes during the lab sessions.

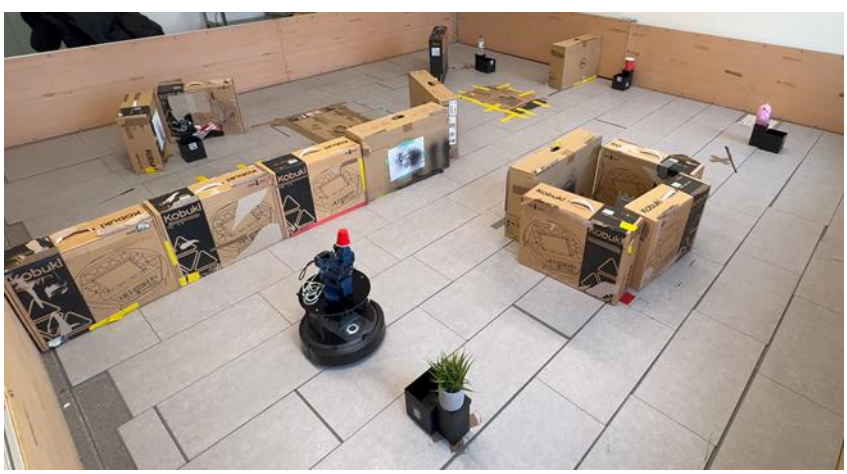


Figure 1: Example Environment Layout with 5 Objects and Multiple Obstacles

- Your team will be provided with: 1) a 2D map of the contest environment generated by slam_toolbox, 2) test locations for the 5 scene objects within that environment, 3) one manipulable object placed on top of the TurtleBot 4's top plate, and its corresponding coordinates.

- The contest has a maximum time limit of 5 minutes. The robot will start at a location in the map of the Instructor/TAs choosing. After the contest starts, the robot needs to first detect the manipulable object using YOLOv8 and pick it up from its top plate. Then, the robot needs to navigate to each of the coordinates provided for the scene objects (5 in total), and run YOLO v8 detection to identify each object. Once the robot identifies which scene object matches its manipulable object, it should place the manipulable object into the garbage bin in front of the scene object, Figure 2. One AprilTag (Figure 3) will be attached to each garbage bin in order to localize the bin. After identifying all scene objects and placing the manipulable object inside the appropriate garbage bin, the robot should go back to its starting position.

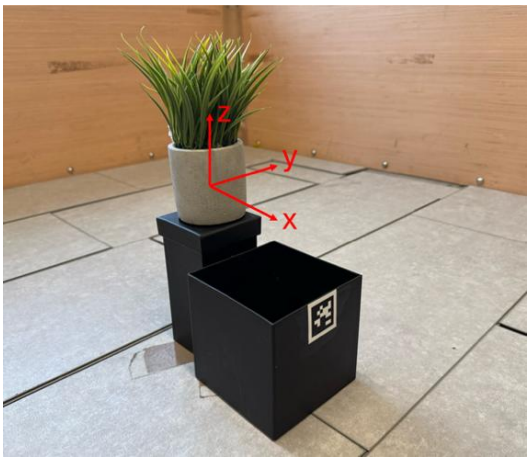

Figure 2. Object Coordinate Frame

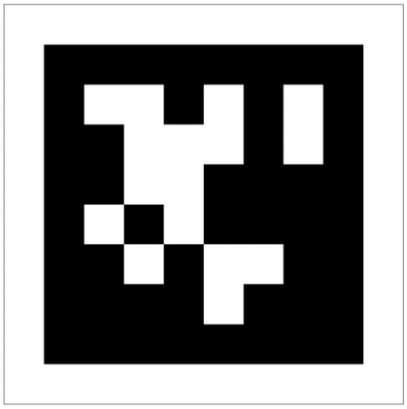
Figure 3. AprilTag Example

- The manipulable object's location on the robot top plate is defined by the coordinates of its centre ($x, y, z$) with respect to the arm base.
- A scene object's location in the environment map is defined by the coordinates of its centre and its orientation ($x,y,\varphi$), where $\varphi$ is about the $z$-axis, with respect to the origin of the given 2D map. The origin of the world coordinate frame is determined by the robot's starting location when the map was created, see Figure 4. The origin is a native property of the map and remains the same during the contest regardless of the starting pose of the robot used for robot localization.
- The TurtleBot 4 does not have to start at the origin of the world coordinate frame when the map is loaded; however, its pose within the map, just like the objects, will be measured with respect to this origin.

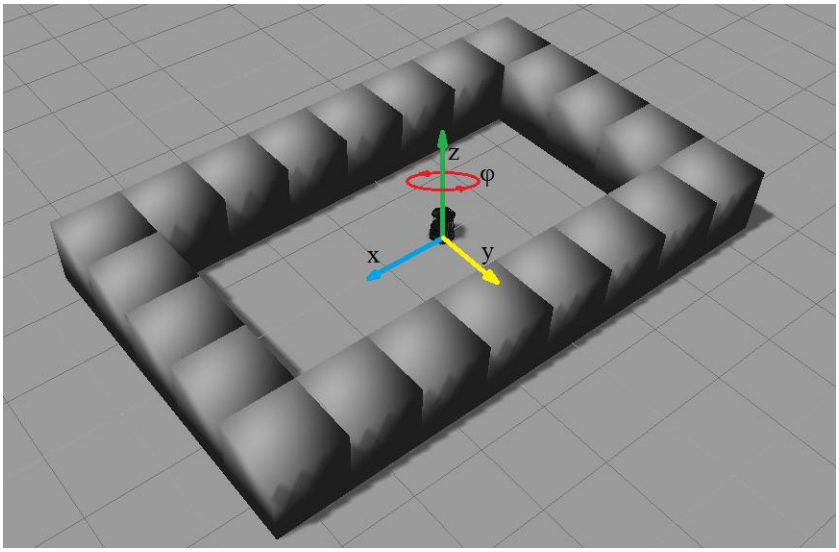

Figure 4: Example World Coordinate Frame

- The object locations are defined by two vectors: 1) a coordinate vector which defines each object's location in *x* and *y*, and 2) an orientation vector which contains the object rotation about the *z* axis. These locations are measured from the object's local frame (Figure 2) with respect to the world coordinate frame (Figure 4) at the origin of the map.
- On contest day, your team will receive a new set of scene object locations and map to utilize for the contest. Please ensure that your code is robust enough to handle such dynamic changes to the locations of the objects.
- Your TurtleBot 4 must utilize the provided navigation library (i.e., nav2) to drive the robot base safely and use the RGB image from the arm wrist camera and the OAK-D Lite camera to perform YOLO detection in order to identify each object. After the scene object that matches the manipulable object is detected, the robot must then identify the ApriTag on the garbage bin in front of the scene object to localize it, and subsequently place the manipulable object in the bin.
- Once the TurtleBot 4 has traversed to all the objects, placed the manipulable object in the bin and returned to its starting location, it must output the following information that will be used for scoring:
    - A file (http://www.cplusplus.com/doc/tutorial/files/) containing: 1) the detected manipulable object name and the corresponding confidence score, and 2) a list of all detected scene objects and their corresponding locations (i.e., object+location).
    - An image of the annotated wrist camera view showing the detection of the manipulable object, including the bounding box and confidence score.
- In order to reduce run time, your team is strongly encouraged to optimize the robot's path to find the objects.

**1.3 Scoring (15%):**

- Your team will be given a total of 2 trials. The best trial will be counted towards your final score.
- The scores for each trial are as follows:
    - Three marks will be provided if your robot arm successfully detects and picks up the manipulable object.

- One mark will be provided for the navigation to each scene object location (5 marks in total).
- One mark will be given if your TurtleBot 4 is able to correctly identify each scene object (5 marks in total).
- Two marks will be given if your TurtleBot 4 can place the manipulable object inside the correct garbage bin.
- One mark will be given if your robot can return to the start location.

**1.4 Procedure:**

Code Development - The following steps should be followed to complete the work needed for this contest:

- Download the mie443_contest2.zip from Quercus. Then extract the contest 2 package (right click > Extract Here) and then move all of the folders (including mie443_contest2, lerobot_controller, lerobot_description, lerobot_moveit) to the ros2_ws/src folder located in your home directory.
- In terminal, change your current directory to ~/ros2_ws using the cd command. Then run

```
colcon build --packages-select mie443_contest2 \
lerobot_description lerobot_controller lerobot_moveit
```

  to compile the code to make sure everything works properly. ****Please note if you do not run this command in the correct directory a new executable of your program will not be created.****
- It is recommended that robot navigation for this contest be handled by the native TurtleBot 4 library called *nav2*. This library takes coordinates and orientations in the global map of the environment and drives the robot to a specified location while handling robot localization and obstacle avoidance. For more information, please refer to the reference material in section A.1.2 in the appendix.
- For object detection, it is recommended to use the YOLOv8 model. Please refer to the reference material in section A.1.3 in the appendix for more information and tutorials with regards to this library.
- Refer to the contest2.cpp and yolo_detector.py file located in the mie443_contest2/src/ folder. For readability and ease of troubleshooting it is important that your team develops the navigation/manipulation code and the object detection code in these respective files.
- **PLEASE DO NOT** make changes to any other files.

Code Breakdown – contest2.cpp

- Sensor Implementation - Pose estimation from AMCL
  - This is the callback function that is run whenever the robot's pose ($x$, $y$, $\varphi$) in the map is published. This callback is found in the robot_pose.cpp file. The variable msg is an object that contains the data pertaining to the robot's current position and orientation.

```
void RobotPose::poseCallback(const geometry_msgs::msg::PoseWithCovarianceStamped::SharedPtr msg) {
    // Extract yaw from quaternion using tf2
    phi = tf2::getYaw(msg->pose.pose.orientation);
    x = msg->pose.pose.position.x;
    y = msg->pose.pose.position.y;
}
```

- Main block and ROS initialization functions.

```
int main(int argc, char** argv) {
    // Setup ROS 2
    rclcpp::init(argc, argv);
    auto node = std::make_shared<rclcpp::Node>("contest2");

    RCLCPP_INFO(node->get_logger(), "Contest 2 node started");

    // Robot pose object + subscriber
    RobotPose robotPose(0, 0, 0);
```

- Definition of the subscriber amclSub that is used to return the global pose of the TurtleBot 4.

```
    auto amclSub = node->create_subscription<geometry_msgs::msg::PoseWithCovarianceStamped>(
        "/amcl_pose",
        10,
        std::bind(&RobotPose::poseCallback, &robotPose, std::placeholders::_1)
    );
```

- Robot Navigation
  - Initialization of the vector containing the object coordinates ($x$, $y$ *and* $\varphi$). If the function is not able to properly initialize the variables it will terminate the program and state what the problem was.

```
    // Initialize box coordinates
    Boxes boxes;
    if(!boxes.load_coords()) {
        RCLCPP_ERROR(node->get_logger(), "ERROR: could not load box coordinates");
        return -1;
    }
```

"`boxes.coords`" is a two-dimensional vector, where the first index of the vector is used to specify which coordinate you would like to choose and the second index is used to select whether you would like to access the $x$ location, $y$ location or orientation of that coordinate, stored in that order. For example, typing `boxes.coords[1][2]` would return the orientation of the second coordinate stored in the vector (vectors are zero-indexed data structures).

Code Breakdown – navigation.cpp

- The `Navigation::`*moveToGoal()* function is used to move the TurtleBot 4 to a goal pose. When destination coordinates ($x$, $y$ and $\varphi$) are passed to the function, it allows the robot to navigate to the commanded location while performing obstacle avoidance using *Nav2*. For more information on TurtleBot 4 with *Nav2* please refer to the reference material in A.1.2 in the appendix.

```
bool Navigation::moveToGoal(double xGoal, double yGoal, double phiGoal) {
```

- Navigation::*moveToGoal()* can be used in contest2.cpp by first constructing a Navigation instance then calling the function with a goal pose:

```
    // Initialize navigation
     Navigation nav(node);

    nav.moveToGoal(goal_x, goal_y, goal_phi);
```

Code Breakdown – coords.xml

- Refer to the coords.xml file located in mie443_contest2/src/boxes_database folder. This file is used to store the coordinate locations that the robot will be traversing. During testing, these locations can be altered to test for robustness but **do not** make changes to the structure of the code for the contest here as a new coords.xml, contest2_env.pgm and contest2_env.yaml will be provided on contest day.
- XML is a coding syntax that allows data to be ordered and stored in a very efficient structure for retrieval in code. Also, because it is XML and not C++ code, it does not need to be complied every time you make a change.
  - The headings in the following block of code contain the *x*, *y* and $\varphi$ information of each coordinate the robot will navigate to with respect to the world coordinate frame of the map. The data in each heading is listed as x, y, $\varphi$ in descending order.

```
    <coordinate1>
    -1.429 <!--x  -->
    1.797  <!--y  -->
    -0.606 <!--phi-->
    </coordinate1>
```

Code Breakdown – yoloInterface.cpp

- The *getObjectName()* function takes two input arguments. The first is which camera to use, the second is whether to save the annotated image of the detection. There are two cameras available on the robot: 1) OAK-D Lite camera on the TurtleBot 4, and 2) the wrist camera on the SO-ARM 101. This function calls another function *captureAndDetect* to first capture an image from the corresponding camera, then run YOLO detection on the captured image.

```
std::string YoloInterface::getObjectName(CameraSource camera, bool save_image) {
    if (captureAndDetect(camera, save_image)) {
        return latest_class_name_;
    }
    return "";
}
```

```
enum class CameraSource {
    OAKD,   // OAK-D camera on TurtleBot4
    WRIST   // Wrist camera on SO-ARM101
};
```

- The *captureAndDetect()* function calls a ROS 2 service to request a single RGB image from the camera.

```
bool YoloInterface::captureAndDetect(CameraSource camera, bool save_image) {
    has_detection_ = false;

    // Select camera
    auto capture_client = (camera == CameraSource::OAKD) ? capture_client_oakd_ :
capture_client_wrist_;
    std::string camera_name = (camera == CameraSource::OAKD) ? "OAK-D" : "Wrist";

    // Capture image
    if (!capture_client->wait_for_service(2s)) {
        RCLCPP_ERROR(node_->get_logger(), "%s camera service not available", camera_name.c_str());
        return false;
    }
```

- Then, it calls another ROS 2 service to run YOLO using the captured RGB image. The detected class name, and confidence score will the stored.

```
    auto detect_request = std::make_shared<mie443_contest2::srv::DetectObject::Request>();
    detect_request->image = capture_response->image;
    detect_request->save_detected_image = save_image;
    auto detect_future = detect_client_->async_send_request(detect_request);

    if (rclcpp::spin_until_future_complete(node_, detect_future, 5s) !=
rclcpp::FutureReturnCode::SUCCESS) {
        RCLCPP_ERROR(node_->get_logger(), "YOLO detection call failed");
        return false;
    }

    auto detect_response = detect_future.get();
    if (!detect_response->success) {
        RCLCPP_WARN(node_->get_logger(), "No detection: %s", detect_response->message.c_str());
        return false;
    }

    // Store results
    latest_class_name_ = detect_response->class_name;
    latest_confidence_ = detect_response->confidence;
    has_detection_ = true;

    RCLCPP_INFO(node_->get_logger(), "Detected: %s (%.2f) from %s camera",
                latest_class_name_.c_str(), latest_confidence_, camera_name.c_str());
```

Code Breakdown – image_capture_server.cpp

- This server runs on the Raspberry Pi of the TurtleBot 4. It subscribes to the OAK-D Lite camera topic and creates a ROS service for capturing the latest RGB image and sends it back to the client once requested.
- A ROS service is used instead of directly subscribing to the camera topic on the laptop. This is due to the fact that subscribing to image topics over Wi-Fi would require the Raspberry Pi to publish every image frame to the laptop, which will overload the Pi, and cause significant delays. By using a service, the Pi transmits only one RGB image when requested, which avoids unnecessary bandwidth usage and prevents the Pi from hanging during high-rate streaming.

```cpp
class ImageCaptureServer : public rclcpp::Node {
public:
    ImageCaptureServer() : Node("image_capture_server") {
        // Subscribe to camera feed to maintain latest image
        image_sub_ = this->create_subscription<sensor_msgs::msg::Image>(
            "/oakd/rgb/preview/image_raw",
            10,
            std::bind(&ImageCaptureServer::imageCallback, this, std::placeholders::_1)
        );
        // Create service server
        service_ = this->create_service<mie443_contest2::srv::CaptureImage>(
            "oakd_camera/capture_image",
            std::bind(&ImageCaptureServer::captureImageCallback, this,
                      std::placeholders::_1, std::placeholders::_2)
        );
```

Code Breakdown – yolo_detector.py

- This file defines a ROS node "yolo_detector", it loads a YOLOv8 model and creates a ROS service "/detect_object". This service will be requested by the *captureAndDetect()* discussed above.

```python
class YoloDetectorNode(Node):
    def __init__(self):
        super().__init__('yolo_detector')

        # Load YOLO model
        self.model = YOLO('yolov8n.pt')
        self.get_logger().info('YOLO model loaded')

        # Confidence threshold
        self.confidence_threshold = 0.5

        # Create service
        self.service = self.create_service(
            DetectObject,
            'detect_object',
            self.detect_callback
        )
```

- The *detect_callback()* method takes the request and returns a response by running YOLO detection. The request and response message are defined in srv/DetectedObject.srv. Based on the response definition, your team should complete the *detect_callback()* method.

```python
    def detect_callback(self, request, response):
        """Process image and return highest confidence detection."""

        # Decode compressed image
        np_arr = np.frombuffer(request.image.data, np.uint8)
        image = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)

        if image is None:
            response.success = False
            response.class_id = -1
            response.class_name = ""
            response.confidence = 0.0
            response.message = "Failed to decode image"
            return response

        # Run YOLO inference
        results = self.model(image, verbose=False, device='cpu')
        boxes = results[0].boxes

        ### YOUR CODE HERE ###

        return response
```

```
# Request
sensor_msgs/CompressedImage image
bool save_detected_image    # Whether to save the detected image to disk
---
# Response
int32 class_id        # COCO class ID (-1 if no detection)
string class_name     # Class name (e.g., "apple", "bottle")
float32 confidence    # Detection confidence (0.0 to 1.0)
bool success          # True if detection successful
```

Code Breakdown – arm_controller.cpp

- This script uses moveit2 for the arm manipulation. Two functions are defined and can be used: *moveToCartesianPose*, and *moveGripper*.
- *moveToCartesianPose* takes a goal pose in cartesian space as input, it uses moveit2 to plan a path and execute the plan. The orientation of the goal pose can be represented as Euler angles or quaternion.
- *moveGripper* takes a goal position as input, it also uses moveit2 for planning and execution.

```
bool ArmController::moveToCartesianPose(double x, double y, double z,
                                        double roll, double pitch, double yaw) {
```

```
bool ArmController::moveToCartesianPose(double x, double y, double z,
                                        double qx, double qy, double qz, double qw) {
```

```
bool ArmController::moveGripper(double position) {
```

Code Breakdown – so101_bridge.py

- This script runs on the Raspberry Pi of the TurtleBot 4. It is in the directory of *lerobot_controller/scripts/*. It defines a ROS 2 node to connect the real SO-ARM101 hardware to ROS 2 using the LeRobot Python API.
- The wrist camera is configured as below, similar to the OAK-D Lite camera, a service is created to capture wrist image.

```
        camera_config = {
            "wrist": OpenCVCameraConfig(
                index_or_path=camera_index,
                fps=30,
                width=640,
                height=480,
            )
        }
```

```
        # Wrist camera capture service
        self.capture_service = self.create_service(
            CaptureImage,
            'wrist_camera/capture_image',
            self.capture_image_callback
        )
```

- The code block below shows how the arm is configured and connected.

```
        # Initialize lerobot robot
        self.config = SO101FollowerConfig(
            port=port,
            id='follower',
            cameras=camera_config,
        )
        self.robot = SO101Follower(self.config)
        self.robot.connect()
        self.get_logger().info("Connected to SO-ARM101 hardware")
```

- Two ROS 2 action servers are created for arm and gripper control. These two action servers will receive the planned trajectory from moveit2 (defined in arm_controller.cpp) and execute the trajectories on the real SO-ARM101 robot arm.

```python
        # Action server for arm trajectory
        self.arm_action_server = ActionServer(
            self,
            FollowJointTrajectory,
            'arm_controller/follow_joint_trajectory',
            execute_callback=self.execute_arm_trajectory,
            goal_callback=self.goal_callback,
            cancel_callback=self.cancel_callback,
            callback_group=self.callback_group
        )

        # Action server for gripper trajectory
        self.gripper_action_server = ActionServer(
            self,
            FollowJointTrajectory,
            'gripper_controller/follow_joint_trajectory',
            execute_callback=self.execute_gripper_trajectory,
            goal_callback=self.goal_callback,
            cancel_callback=self.cancel_callback,
            callback_group=self.callback_group
        )
```

Code Breakdown – apriltag_detector.cpp

- The apriltag_ros package will be run on the Pi to detect whether an AprilTag is in the camera's field of view. It will output the pose of the tag. The apriltag_detector script listens to the tf transforms published by the apriltag_ros package for localization.
- The function *getVisibleTags* takes a list of tag candidates to check if any of the tags are detected by the OAK-D camera. It calls another function *isTagVisible* for each tag.

```cpp
std::vector<int> AprilTagDetector::getVisibleTags(const std::vector<int>& candidate_ids, int
timeout_ms) {
    std::vector<int> visible_tags;

    for (int tag_id : candidate_ids) {
        if (isTagVisible(tag_id, timeout_ms)) {
            visible_tags.push_back(tag_id);
        }
    }

    return visible_tags;
}
```

- The function *isTagVisible* looks up the latest transform between the reference frame and the tag frame. If the latest transform is not recent, or if the transform does not exist, it means the tag is not detected.

```cpp
bool AprilTagDetector::isTagVisible(int tag_id, int timeout_ms) {
    (void)timeout_ms;  // Not used - we check staleness instead
    std::string tag_frame = getTagFrameName(tag_id);

    try {
        // Get latest cached transform (non-blocking)
        auto transform = tf_buffer_->lookupTransform(
            reference_frame_,
            tag_frame,
            tf2::TimePointZero
        );

        // Check if transform is recent (within 500ms)
        auto transform_age = node_->get_clock()->now() - transform.header.stamp;
        if (transform_age.seconds() > 0.5) {
            return false;  // Transform is stale - tag no longer visible
        }

        return true;
    } catch (const tf2::TransformException& ex) {
        // Transform not available - tag never seen
        return false;
    }
}
```

- The function *getTagPose* can be called to return the pose between the detected tag frame and the reference frame. The reference frame could be set up as the base frame (base_link) or world frame (map).

```cpp
std::optional<geometry_msgs::msg::Pose> AprilTagDetector::getTagPose(int tag_id, int timeout_ms) {
    auto transform_opt = getTagTransform(tag_id, timeout_ms);

    if (!transform_opt.has_value()) {
        return std::nullopt;
    }

    // Convert TransformStamped to Pose
    geometry_msgs::msg::Pose pose;
    pose.position.x = transform_opt->transform.translation.x;
    pose.position.y = transform_opt->transform.translation.y;
    pose.position.z = transform_opt->transform.translation.z;
    pose.orientation = transform_opt->transform.rotation;

    return pose;
}
```

Compiling

- Every time the code is changed you must compile it from terminal in the ros2_ws directory using the following command. If you do not do this, a new executable will not be created when you run it.

```
colcon build --packages-select mie443_contest2
```

Robot Base Simulation - To simulate the TurtleBot 4 base before implementing your code on the physical TurtleBot 4, the following steps should be followed:

- Start Gazebo
  - Begin by launching the simulated world through the following commands (the available worlds include warehouse, maze, and depot):

```
ros2 launch turtlebot4_gz_bringup turtlebot4_gz.launch model:=lite
world:=maze
```

- To run the amcl localization and obstacle avoidance algorithm in simulation, use the following command (make sure the map name corresponds to the world):

```
ros2 launch turtlebot4_navigation localization.launch.py
map:=/opt/ros/jazzy/share/turtlebot4_navigation/maps/maze.yaml
```

- To visualize robot information (i.e., position, sensor reading), run the following command in a separate terminal window:

```
ros2 launch turtlebot4_viz view_navigation.launch.py
```

- All other commands to view and save the map are the same as in the amcl tutorial.
  - Before running your code, make sure the information contained in ../boxes_database/coords.xml is for the map which you launched in the previous step. If not, copy the correct information to ../boxes_database/coords.xml. For Gazebo, xml files corresponding to the maps are located in ../maps folder. Run the following command to start running your code in simulation:

```
ros2 run mie443_contest2 contest2
```

Robot Arm Simulation - To simulate the SO-ARM 101 before implementing your code on the real arm, follow the next steps:

- Start Gazebo and load the arm:

```
ros2 launch lerobot_description so101_gazebo.launch.py
```

- In a new terminal, start the arm controller by running:

```
ros2 launch lerobot_controller so101_controller.launch.py
```

- In a new terminal, start rviz2 and moveit2 by running:

```
ros2 launch lerobot_moveit so101_moveit.launch.py
```

- Start your code by running:

```
ros2 run mie443_contest2 contest2
```

Robot Execution - When you are ready to run your program on the TurtleBot 4, the following steps should be followed:

- If your team chooses to test in a different area than the contest environment, then the default map **will not work** and a new map must be made for the robot. To do this, please complete the following steps:
    - Follow the mapping tutorial in the reference material section A.1.2 in the appendix.
    - Copy the resulting my_map.yaml, and my_map.png files to the maps folder in your code package at the address: ../mie443_contest2/maps.
    - Change test destination coordinates in your coords.xml file to reflect the changes in the map.
- Place the TurtleBot 4 on the dock if it is in storage mode (when all lights are off) to turn on the robot.
- Open a terminal, run the undock command below or manually remove the robot from the dock. If you run the undock command, the robot would undock itself.

```
ros2 action send_goal /undock irobot_create_msgs/action/Undock {}
```

- In a new terminal, run the following command to start AMCL localization.

```
ros2 launch turtlebot4_navigation localization.launch.py
map:=<map_path>/<map_name>.yaml
```

- In a new terminal, run the following command to start nav2 for navigation.

```
ros2 launch turtlebot4_navigation nav2.launch.py
```

- In a new terminal, start rviz2 for visualization by running the following command. It also allows the user to troubleshoot the navigation algorithm by providing an interface to publish navigation goals for the TurtleBot 4 to pursue.

```
ros2 launch turtlebot4_viz view_navigation.launch.py
```

- To start the OAK-D Lite camera image capture service on the TurtleBot 4, follow these steps:
    - Start a new terminal and ssh into the TurtleBot 4:

      ```
      ssh ubuntu@<ip>
      ```

      If you don't know the IP address, run the following command in a new terminal on the laptop:

      ```
      ros2 topic echo /ip
      ```

    - After ssh, run the following command to start the image capture server. **This command must run on the Pi.**

      ```
      ros2 run mie443_contest2 image_capture_server
      ```

- To start YOLO detector, follow these steps:
    - On the laptop, start a new terminal, and activate the virtual environment for Python

      ```
      source contest2/bin/activate
      ```

    - In the same terminal, start the YOLO detector node

      ```
      ros2 run mie443_contest2 yolo_detector.py
      ```

- Make sure the SO-ARM101 is connected:
    - The wrist camera cable should be connected to the TurtleBot 4.
    - The USB-C to USB-A cable should be connected from the arm controller board to the TurtleBot 4.

- The battery should be connected to the arm controller board.

- To start the arm,
  - Start a new terminal and ssh into the TurtleBot 4:

```
ssh ubuntu@<ip>
```

  - After ssh, activate the virtual python environment, then start the arm in the same terminal. **These commands must run on the Pi.**

```
source contest2/bin/activate
```

```
ros2 launch lerobot_moveit so101_turtlebot.launch.py
```

  - Start a new terminal on the laptop, and run the following command to start moveit2. **This command must run on the laptop.**

```
ros2 launch lerobot_moveit so101_laptop.launch.py
```

- Start a new terminal and ssh into the TurtleBot 4

```
ssh ubuntu@<ip>
```

  Run the following command to start AprilTag detection.

```
ros2 launch apriltag_ros camera_36h11.launch.yml
```

- Before running your code, make sure the information contained in ../boxes_database /coords.xml is for the map which you launched above. If not, copy the correct information to ../boxes_database /coords.xml. To execute your code, run the following line in its own terminal:

```
ros2 run mie443_contest2 contest2
```

- After the robot has completed the pick and place task, object detection, and returned to its starting location, it ****MUST**** output what manipulable object and scene objects have been found as well as their respective coordinate index for evaluation. Please ensure you provide everything requested in Section 1.2, namely:
  - A file (http://www.cplusplus.com/doc/tutorial/files/) containing: 1) the detected manipulable object name and the corresponding confidence score, and 2) a list of all detected scene objects and their corresponding locations (i.e., object+location).
  - An image of the annotated wrist camera view showing the detection of the manipulable object, including the bounding box and confidence score.
- Cancel all processes by pressing Ctrl-C in their respective terminals when you are done.

**1.5 Report:**

- The report for each contest is worth half the marks and should provide detailed development and implementation information to meet contest objectives (15 marks).
- Marking Scheme:
  - The problem definition/objective of the contest. (1 mark)
    - Requirements and constraints
  - Strategy used to motivate the choice of design and winning/completing the contest within the requirements given. (2 marks)
  - Detailed Robot Design and Implementation including:
    - Sensory Design (4 marks)

            - Sensors used, motivation for using them, and how are they used to meet the requirements including functionality.
        - Controller Design, both low-level and high-level control (including architecture and all algorithms developed) (5 marks)
            - Architecture type and design of high-level controller used (adapted from concepts in lectures)
            - Low-level controller
            - All algorithms developed and/or used
            - Changes and additions to the existing code for functionality
    - Future Recommendations (1 mark)
        - What would you do if you had more time?
        - Would you use different methods or approaches based on the insight you now have?
    - Full C++ ROS 2 code (in an appendix). Please do not put full code in the text of your report. (2 marks)
    - Contribution of each team member with respect to the tasks of the particular contest (robot design and report). You can use an Attribution Table for this. (Towards Participation Marks)
    - The contest package folder containing your code will also be submitted alongside the softcopy (PDF version) of your report. (Towards the Code Marks Above)
- Your report should be a maximum of 20 pages, single spaced, font size 12 (not including appendix).

**1.6 Reference Materials for the Contest**:

The following materials in Appendix A will be of use for Contest 2:

- A.1.1
- A.1.2
- A.1.3
- A.1.4